Asymptotic analysis

CS3230 · Lecture 1

Demo course material, written for OpenPapr

Why count steps?

- Wall-clock time depends on the machine and the compiler
- Count basic operations instead
- Focus on how the count grows as n becomes large

Input size and cases

- n = number of items, bits or vertices
- Worst case, best case, average case
- We usually analyse the worst case

Growth rates

- $1 < \log n < n < n \log n < n^2 < 2^n < n!$
- Constant factors are dropped
- Lower-order terms are dropped

Big-O

- f(n) = O(g(n)) if there are c > 0 and n0 with f(n) <= c*g(n) for all n >= n0
- An upper bound on growth
- Example: 3n^2 + n is O(n^2), with c = 4 and n0 = 1

Big-Omega

- f(n) = Omega(g(n)) if f(n) >= c*g(n) for all n >= n0
- A lower bound on growth
- Example: n log n is Omega(n)

Big-Theta

- f(n) = Theta(g(n)) if f is both O(g) and Omega(g)
- A tight bound
- Example: 5n + 2 is Theta(n)
- Theta is a claim about BOTH bounds

The three bounds

- O(g): grows no faster than g
- Omega(g): grows at least as fast as g
- Theta(g): both at once

Proving a bound

- Pick constants c and n0
- Show the inequality for every n >= n0
- Common mistake: letting c depend on n

Little-o and little-omega

- Strict versions of O and Omega
- f = o(g): f grows strictly slower than g
- n = o(n^2), but n is not o(n)

Comparing growth rates

- Take the limit of f(n)/g(n) as n grows
- 0 means f = o(g); a positive constant means Theta
- Infinity means f = omega(g)

Logarithms

- log_a n = Theta(log_b n) for constants a, b > 1
- So the base does not matter inside O
- n^0.001 eventually beats log n

Polynomials vs exponentials

- $n^k = o(2^n)$ for any constant k
- $2^n = o(3^n)$
- $n!$ grows faster than 2^n

Sums

- 1 + 2 + ... + n = Theta(n^2)
- 1 + 1/2 + ... + 1/n = Theta(log n)
- A geometric sum is dominated by its largest term

Summary

- Count operations, not seconds
- Use O, Omega and Theta precisely
- Next: recurrences